

Phase 0: Programming Foundations, Rebuilt

Full specifications for projects 1 to 18

escape-velocity | Stage 1 | Months 1 to 6

How to use these specifications

Phase 0 is not a warm-up. These eighteen projects together build evkit, the installable toolkit that every project in Phases 4 through 11 imports. When you write an orbit propagator in project 128, it will import your units, your time scales, your frames and your TLE parser. When you validate it in project 129, it will import your Horizons reader. Nothing here is throwaway.

That is the difference from the curriculum this replaces. A contact book teaches dictionaries and is then deleted. A TLE catalogue manager teaches the same dictionaries and is still in use three years later.

The working method

- Read the specification. Write the definition of done in your own words before writing any code.
- Work through the concepts list first. Do not start coding while the concept is still fuzzy, and do not read for more than about an hour before starting.
- Build the smallest thing that does something, then extend. A Quantity class that only handles metres is a valid first commit.
- Write the tests as you go, not at the end. Project 11 is where the suite becomes comprehensive, but every module ships with tests from project 1.
- When it works, write the docstring for someone who has never seen it. If that is hard to write, the API is probably wrong.
- Commit with a real message. Update the build log in the README.

A note on the no-AI-code rule

These specifications tell you what to build and where to learn it. They deliberately contain no code. Use an assistant to explain a concept, to review something you wrote, or to tell you why an error message means what it means. Do not use one to produce the implementation. Phase 0 is where the habits form, and a toolkit you did not write is a toolkit you cannot debug at two in the morning during Phase 5.

Order

Projects 1 to 4 are strictly sequential and each depends on the one before. Projects 5 to 10 can be done in any order once 1 to 4 exist. Projects 11 and 12 should be started early and grown continuously rather than done as a block. Projects 13 to 15 are environment work and can be slotted into any week where you want a change of pace. Projects 16 to 18 are data access and are best done immediately before you need them, which for most people is around project 128.

Reference texts worth acquiring now

- Vallado, Fundamentals of Astrodynamics and Applications. The standard reference for Phases 5 and 6. Expensive, and the single most useful book on this list.
- Curtis, Orbital Mechanics for Engineering Students. More accessible than Vallado and a better first read.
- Taylor, An Introduction to Error Analysis. Short, cheap, and the basis of project 2.
- MIT's Missing Semester course, free online, which covers most of projects 13 to 15.

Project 1: Scientific units and dimensions library

What you are building

A Python class that carries a numeric value together with its physical dimensions, so that adding metres to seconds raises an error instead of silently producing nonsense. This is the first module of `evkit`, the toolkit you will import for the next three years.

Why it matters

The Mars Climate Orbiter was lost in 1999 because one team used pound-seconds and another used newton-seconds. Unit errors are the most expensive class of bug in aerospace software. Building your own unit system teaches you why libraries like `astropy.units` and `pint` exist, and you will understand their design instead of just calling them.

Requirements

- A Quantity class holding a float value and a dimension vector over the seven SI base dimensions: length, mass, time, current, temperature, amount, luminous intensity
- Arithmetic through operator overloading: `__add__`, `__sub__`, `__mul__`, `__truediv__`, `__pow__`, `__neg__`
- Addition and subtraction raise a `DimensionError` when dimensions do not match
- Multiplication and division add and subtract dimension vectors correctly
- Powers work with integer and rational exponents, so that a square root of an area gives a length
- Conversion between units of the same dimension: metres to kilometres, kilometres per hour to metres per second
- A registry of named units built from the base set: newton, joule, watt, pascal, astronomical unit, light year, parsec
- `__repr__` and `__str__` that print the value with its unit legibly
- Comparison operators that respect dimensions
- No third party dependencies. Standard library only

Concepts to learn

- Operator overloading and dunder methods: <https://docs.python.org/3/reference/datamodel.html#special-method-names>
- Dataclasses and immutability: <https://docs.python.org/3/library/dataclasses.html>
- Custom exceptions: <https://docs.python.org/3/tutorial/errors.html#user-defined-exceptions>
- Fractions for rational exponents: <https://docs.python.org/3/library/fractions.html>
- SI base units and dimensional analysis: <https://www.bipm.org/en/measurement-units>

Where to find the material

- Read the `astropy.units` documentation before you start, then close it and build your own: <https://docs.astropy.org/en/stable/units/>
- The `pint` library documentation for an alternative design: <https://pint.readthedocs.io/>
- Real Python on operator overloading: <https://realpython.com/operator-function-overloading/>

Definition of done

- `Quantity(9.81, 'm/s2') * Quantity(70, 'kg')` returns a force in newtons with the right dimensions
- `Quantity(1, 'm') + Quantity(1, 's')` raises `DimensionError` with a clear message
- A test suite covers every operator, every base dimension, and at least six error cases
- The module sits in `toolkit/src/evkit/units.py` and imports cleanly
- Written up in the README of the toolkit with three usage examples

Edge cases to handle

- Adding two quantities with the same dimension but different units, for example metres and kilometres. Convert, do not reject
- Multiplying by a bare number, which should work and leave dimensions unchanged
- Dividing two identical dimensions, which should produce a dimensionless quantity rather than an error
- Zero and negative values
- Fractional powers that produce non-integer dimension exponents
- Very large and very small magnitudes, where float precision starts to matter

Stretch, only after done is met

- Add non-SI unit systems: CGS, imperial, and the astronomical system where G equals 1
- Parse unit strings like 'kg m² / s³' with a small tokeniser rather than a lookup table
- Add uncertainty propagation, which becomes project 2
- Add a `__format__` method so that f-strings can control precision and unit display

Project 2: Physical constants module with uncertainties

What you are building

A module of physical constants, each carrying a value, a standard uncertainty, and a unit from project 1. Arithmetic on these constants propagates the uncertainty correctly.

Why it matters

Every measured number in science has an uncertainty, and a result quoted without one is not a scientific result. Uncertainty propagation is the single most common thing physicists do that programmers never learn. It also shows up again in project 218 when you fit transit parameters and in every validation plot you make.

Requirements

- Extend the Quantity class from project 1 into a Measurement class with a nominal value and a standard uncertainty
- Implement first-order error propagation for addition, subtraction, multiplication, division and powers
- Handle correlated versus independent variables explicitly, and document which you assume
- Load the CODATA recommended values for at least twenty constants: G , c , h , $\hbar$, k_B , N_A , m_e , m_p , e , ϵ_0 , μ_0 , σ , R , g_0 , and the astronomical set GM_{sun} , GM_{earth} , R_{earth} , AU, parsec, solar luminosity
- Store constants in a data file rather than hardcoding them in Python, so they can be updated
- Format output as value plus or minus uncertainty, with the uncertainty rounded to two significant figures and the value matched to it
- Report relative uncertainty on request

Concepts to learn

- Propagation of uncertainty: https://en.wikipedia.org/wiki/Propagation_of_uncertainty
- Significant figures and rounding rules for uncertainties
- Reading structured data files: <https://docs.python.org/3/library/json.html> and <https://docs.python.org/3/library/tomllib.html>
- The uncertainties package design, as a reference: <https://pythonhosted.org/uncertainties/>

Where to find the material

- NIST CODATA fundamental physical constants: <https://physics.nist.gov/cuu/Constants/>
- IAU nominal values for solar and planetary constants, resolution B3 of 2015
- Taylor, An Introduction to Error Analysis, chapters 3 and 4. The standard undergraduate text

Definition of done

- Computing the escape velocity of Earth from GM and R gives 11.19 km/s with a propagated uncertainty
- Every constant matches its CODATA value to the last quoted digit, verified by a test
- Formatting produces 6.674 30 plus or minus 0.000 15 times 10 to the minus 11, not fifteen unrounded digits
- Tests cover propagation through each operator against hand-computed values

Edge cases to handle

- A constant with zero uncertainty, such as the exactly defined speed of light
- Subtracting two nearly equal quantities, where relative uncertainty explodes. This is worth a comment in the code
- Powers with negative exponents
- Division by a quantity whose uncertainty range includes zero

Stretch, only after done is met

- Monte Carlo uncertainty propagation as a cross-check on the analytical formulas, and a comparison of when they disagree
- Correlation matrix support for correlated inputs
- A command line tool that evaluates an expression with constants and prints the result with uncertainty

Project 3: Time systems converter

What you are building

A module that converts between UTC, TAI, GPS time, terrestrial time, Julian Date, Modified Julian Date, and Greenwich Mean Sidereal Time, handling leap seconds correctly.

Why it matters

Time is the single most common source of silent error in space software. UTC has leap seconds, TAI does not, GPS time is offset from both, and sidereal time governs where a telescope or an antenna actually points. Your orbit propagator in project 128 is worthless if its epoch handling is wrong, and the error will look like a physics bug rather than a time bug.

Requirements

- Represent an instant internally in a continuous scale, TAI or Julian Date, rather than in UTC
- Convert UTC to and from TAI using a leap second table
- Convert to and from GPS time, which is TAI minus 19 seconds
- Convert to and from terrestrial time, which is TAI plus 32.184 seconds
- Compute Julian Date and Modified Julian Date from a calendar date and time
- Compute Greenwich Mean Sidereal Time from UT1, with the standard polynomial
- Parse and emit ISO 8601 strings
- Load the leap second table from a data file, and state the date it was last updated
- Sub-second precision maintained across all conversions

Concepts to learn

- Leap seconds and why UTC is discontinuous: https://en.wikipedia.org/wiki/Leap_second
- Julian Date: https://en.wikipedia.org/wiki/Julian_day
- Sidereal time: https://en.wikipedia.org/wiki/Sidereal_time
- Python datetime, and its limitations for scientific use: <https://docs.python.org/3/library/datetime.html>
- Floating point precision and why a single float cannot hold a Julian Date to microsecond precision

Where to find the material

- IERS Bulletin C, the authoritative leap second announcement: <https://www.iers.org/>
- Vallado, Fundamentals of Astrodynamics and Applications, chapter 3. This is the reference text for the whole astrodynamics phase and worth acquiring now
- The astropy.time documentation, as a design reference: <https://docs.astropy.org/en/stable/time/>

Definition of done

- Every conversion round-trips to within one microsecond, verified by tests
- Your Julian Date for 2000 January 1 at 12:00 TT equals 2451545.0 exactly. This is the J2000 epoch and it must be exact
- A conversion across the 2016 December 31 leap second gives the right answer, and there is a test for it
- GMST for a known date matches a published value to within a millisecond
- The leap second table is in a data file with its source and date documented

Edge cases to handle

- Dates before 1972, when UTC was defined differently and leap seconds did not exist
- The leap second itself, 23:59:60, which datetime cannot represent
- Dates far in the future, where the leap second table is unknown. Warn rather than silently extrapolate

- Negative Julian Dates for dates before 4713 BC
- Precision loss when a Julian Date is stored as a single 64-bit float. Consider storing it as two floats

Project 4: Coordinate frame converter

What you are building

A module that transforms position and velocity vectors between Earth-centred inertial, Earth-centred Earth-fixed, geodetic latitude and longitude and altitude, and topocentric azimuth, elevation and range as seen from a ground station.

Why it matters

Every satellite question reduces to a frame question. Where is it, relative to what, at what time. This module is what lets your propagator answer where is the ISS right now over Kigali, and it is the direct prerequisite for the pass prediction in project 133 and the ground station in project 283.

Requirements

- Rotation matrices about each axis, composed correctly
- ECI to ECEF and back, using Greenwich sidereal time from project 3
- ECEF to geodetic latitude, longitude and altitude on the WGS84 ellipsoid, and back
- ECEF to topocentric south-east-zenith or east-north-up, and then to azimuth, elevation and range
- Velocity transformation including the Earth rotation term, which is where most implementations go wrong
- All angles handled in radians internally, degrees only at the interface
- Use the units from project 1 on every public function signature

Concepts to learn

- Earth-centred inertial and Earth-fixed frames: https://en.wikipedia.org/wiki/Earth-centered_inertial
- Geodetic versus geocentric latitude, and why they differ: https://en.wikipedia.org/wiki/Geodetic_coordinates
- WGS84 ellipsoid parameters: https://en.wikipedia.org/wiki/World_Geodetic_System
- Rotation matrices: https://en.wikipedia.org/wiki/Rotation_matrix
- The velocity transformation term $\omega \times r$, and why forgetting it produces a 460 m/s error at the equator

Where to find the material

- Vallado, Fundamentals of Astrodynamics and Applications, chapter 3, sections on coordinate systems
- Curtis, Orbital Mechanics for Engineering Students, chapter 5
- The pyproj library, for cross-checking your geodetic conversions: <https://pyproj4.github.io/pyproj/>

Definition of done

- ECEF to geodetic and back round-trips to within one millimetre
- The geodetic conversion is iterative or uses a closed-form method, and you can state which and why
- Azimuth and elevation of a known object from a known ground station match a published value to within 0.01 degrees
- Velocity transformation tested against a case with a nonzero Earth rotation contribution
- Cross-checked against pyproj for at least twenty random points

Edge cases to handle

- Points at the poles, where longitude is undefined
- Points at the geocentre, where the geodetic conversion is singular
- Altitude below the ellipsoid surface, which is valid and occurs in real terrain
- Azimuth wrapping across 360 degrees
- Elevation below the horizon, which is a valid negative number, not an error

Stretch, only after done is met

- Add the true equator mean equinox frame and the precession and nutation corrections, which becomes project 127
- Add polar motion from IERS Earth orientation parameters
- Plot a ground track over a world map to see the conversion working visually

Project 5: TLE parser and catalogue manager

What you are building

A parser for the two-line element format used by NORAD and Celestrak to distribute orbital data for every tracked object, plus a catalogue class that stores, indexes and queries thousands of them.

Why it matters

TLEs are how the entire world publishes satellite orbits. Your propagator, your debris analysis in project 285, your ground station, and your pass predictions all consume this. The format is old, fixed-width, and unforgiving, which makes it an excellent lesson in defensive parsing.

Requirements

- Parse both lines into a structured object: catalogue number, classification, international designator, epoch, mean motion and its first and second derivatives, drag term, inclination, right ascension of ascending node, eccentricity, argument of perigee, mean anomaly, revolution number
- Handle the implied decimal point in the eccentricity field, which has no leading zero
- Handle the exponential notation in the drag and second derivative fields
- Verify the modulo 10 checksum on each line and reject lines that fail
- Convert the two-digit epoch year and fractional day of year into a proper time object from project 3
- A Catalogue class that loads a full Celestrak file, indexes by catalogue number and by name, and supports queries by inclination band, altitude band and epoch age
- Round-trip: emit a TLE from a parsed object and get the original string back byte for byte

Concepts to learn

- The two-line element format specification: <https://celestrak.org/columns/v04n03/>
- Mean orbital elements versus osculating elements, and why TLEs are not instantaneous positions
- Fixed-width parsing and why `str.split` is the wrong tool here
- Checksums: <https://en.wikipedia.org/wiki/Checksum>

Where to find the material

- Celestrak, the primary public source: <https://celestrak.org/NORAD/elements/>
- Space-Track, which requires a free account and has the full catalogue: <https://www.space-track.org/>
- The `python-sgp4` package, which you will compare against in project 130: <https://pypi.org/project/sgp4/>

Definition of done

- Parses the full Celestrak active satellite file, currently several thousand objects, without a single unhandled exception
- Round-trip emission is byte-identical for every TLE in that file
- Checksum verification catches a deliberately corrupted line
- Epoch conversion for a known TLE matches the value reported by `python-sgp4`
- Loading and indexing the full catalogue takes under two seconds

Edge cases to handle

- Lines shorter than 69 characters, which occurs in some feeds
- Names longer than 24 characters in the optional title line
- Epoch years in the 1957 to 1999 range, where the two-digit year rule flips
- Negative values in the drag term
- Eccentricity of exactly zero, and values very close to one

- Duplicate catalogue numbers with different epochs, which is normal and means you must keep the latest
- Files with Windows line endings

Stretch, only after done is met

- Fetch directly from Celestrak with caching, which anticipates project 16
- Detect and flag TLEs that are stale, more than a few days old, since accuracy degrades quickly
- Store the catalogue in SQLite and compare query performance against the in-memory index

Project 6: Ephemeris reader for JPL Horizons output

What you are building

A parser for the text output of NASA JPL's Horizons system, which provides high precision positions and velocities for every major solar system body and spacecraft.

Why it matters

This is the module that makes validation possible. When you build a propagator in project 128, you need something authoritative to compare it against, and Horizons is that authority. Without this module, project 129, the single most important upgrade to your existing work, cannot happen.

Requirements

- Parse the Horizons vector table output format, which delimits data between SOE and EOE markers
- Extract Julian Date, position vector and velocity vector for each record
- Parse the header to capture the target body, centre body, reference frame and units
- Return the data as a structured object with a time array and position and velocity arrays
- Interpolate between records so a position can be requested at any time in the span, using the cubic spline you will later formalise in project 74
- Handle both the vector table and the observer table formats, or clearly document that you support one
- Store fetched ephemerides on disk so you do not re-download for every test run

Concepts to learn

- The Horizons system and its output formats: <https://ssd.jpl.nasa.gov/horizons/>
- The Horizons API documentation: <https://ssd-api.jpl.nasa.gov/doc/horizons.html>
- Reference frames: ICRF, ecliptic and equatorial, and which one your request returns
- Robust text parsing with regular expressions: <https://docs.python.org/3/library/re.html>

Where to find the material

- JPL Horizons web interface, for generating test files by hand first: <https://ssd.jpl.nasa.gov/horizons/app.html>
- The astroquery.jplhorizons module, as a reference implementation to compare against: <https://astroquery.readthedocs.io/>
- SPICE and the SPK kernel format, which is the professional alternative and worth knowing exists: <https://naif.jpl.nasa.gov/naif/>

Definition of done

- Fetches and parses the Earth ephemeris relative to the solar system barycentre for a full year
- Interpolated positions between records agree with directly requested Horizons values to better than one kilometre
- Header metadata is captured and exposed, so you always know what frame your numbers are in
- A cached file is reused rather than refetched, verified by a test
- Works for at least three targets: a planet, the Moon, and a spacecraft

Edge cases to handle

- Requests that return an error page rather than data
- Time spans that cross the limits of the underlying ephemeris
- Different units in the header, kilometres versus astronomical units
- Records with fewer fields than expected
- Very large files, tens of thousands of records, where line by line parsing matters

- Interpolation requested outside the fetched span, which must raise rather than extrapolate silently

Project 7: Telemetry log analyser

What you are building

A tool that ingests a time-stamped multi-channel telemetry log, computes per-channel statistics, detects gaps and outliers, and produces a summary report.

Why it matters

Real instrument data arrives as a messy time series with missing samples, dropouts, and occasional nonsense values. Learning to characterise it before analysing it is a habit that pays off in every phase after this one, particularly in Phase 6 fault detection and Phase 11 light curve work.

Requirements

- Read a CSV or similar log with a timestamp column and multiple numeric channels
- Per channel, compute count, mean, standard deviation, minimum, maximum, and the 5th, 50th and 95th percentiles
- Use Welford's online algorithm for the mean and variance rather than a naive two-pass sum
- Detect gaps: intervals where the sampling period exceeds the nominal period by more than a tolerance
- Detect outliers using the interquartile range method, and report them rather than removing them
- Detect stuck channels, where the value does not change for a suspiciously long run
- Compute the duty cycle and the effective sample rate per channel
- Emit a text or markdown summary report

Concepts to learn

- The csv module: <https://docs.python.org/3/library/csv.html>
- Welford's online algorithm for variance: https://en.wikipedia.org/wiki/Algorithms_for_calculating_variance
- The interquartile range and outlier detection: https://en.wikipedia.org/wiki/Interquartile_range
- Why the naive variance formula loses precision, and what catastrophic cancellation means

Where to find the material

- Use a real dataset rather than generating one. Options: NOAA space weather archives at <https://www.swpc.noaa.gov/>, or any public sensor dataset
- Knuth, The Art of Computer Programming volume 2, section 4.2.2, where Welford's method is described
- The pandas describe method, as a target for what your output should contain

Definition of done

- Runs on a real log of at least one hundred thousand rows
- Your Welford variance agrees with the naive two-pass result on well-conditioned data and beats it on a deliberately ill-conditioned test case
- Gap detection finds a gap you introduced deliberately
- The report is readable by someone who has never seen the data
- Handles a file that does not fit comfortably in memory by streaming

Edge cases to handle

- Missing values represented as empty strings, NaN, or sentinel values like minus 999
- Timestamps out of order, or duplicated
- A channel that is entirely missing
- Mixed types within a column
- Time zone information present in some rows and absent in others

- A file with no data rows at all

Project 8: Structured logging and diagnostics module

What you are building

A configured logging setup that every later project imports, producing consistent, level-controlled, structured output to both console and file.

Why it matters

Print statements do not survive contact with a long simulation. When your N-body run takes four hours and produces wrong answers, the log is the only evidence you have. Getting this right once, early, is what separates projects you can debug from projects you have to rerun.

Requirements

- Configure the standard library logging module rather than writing your own
- Console handler at INFO level with a readable format, file handler at DEBUG level with full detail
- Include timestamp, level, module name and line number in the file format
- A structured mode that emits JSON lines, for logs that will be parsed later
- A context manager or decorator that logs the entry, exit and duration of a function
- A progress reporter for long loops that logs at a fixed time interval rather than every iteration
- Log rotation so a long run does not fill the disk
- A single import that any project can use: `from evkit.logging_setup import get_logger`

Concepts to learn

- The logging module and its hierarchy: <https://docs.python.org/3/library/logging.html>
- The logging cookbook, which is genuinely the best documentation Python has: <https://docs.python.org/3/howto/logging-cookbook.html>
- Context managers: <https://docs.python.org/3/library/contextlib.html>
- Decorators: <https://realpython.com/primer-on-python-decorators/>

Where to find the material

- The logging HOWTO: <https://docs.python.org/3/howto/logging.html>
- structlog, for a view of what structured logging looks like at scale: <https://www.structlog.org/>

Definition of done

- Every module written from project 9 onward uses it, with no print statements anywhere in evkit
- A log file from a run of project 7 is readable and complete
- The timing decorator reports durations that match a stopwatch
- Level filtering works: setting the console to WARNING silences INFO without touching the file output
- Rotation triggers correctly on a deliberately small size limit

Edge cases to handle

- Logging from multiple modules, where the hierarchy matters
- Logging inside a library that a user might configure differently. Never call `basicConfig` inside library code
- Unicode in log messages
- Exceptions logged with full tracebacks using `exc_info`
- Very high frequency logging, which will dominate runtime if you are not careful

Project 9: Experiment configuration system

What you are building

A system that loads run parameters from a YAML or TOML file, validates them against a schema, applies defaults, and records the exact configuration alongside every set of results.

Why it matters

Reproducibility begins here. Six months from now you will look at a figure and need to know what timestep produced it. If the answer lives in a variable you edited by hand, the figure is worthless. This module is also the direct prerequisite for the reproducible repository in project 90 and the DVC pipeline in project 88.

Requirements

- Load configuration from TOML using the standard library `tomllib`, or YAML if you prefer
- Validate against a declared schema: required keys, types, allowed ranges
- Apply documented defaults for anything optional
- Allow command line overrides of any config value, so sweeps do not require editing files
- Compute a hash of the resolved configuration, and use it to name the output directory
- Write the resolved configuration, including defaults and overrides, into the output directory
- Record the git commit hash and whether the working tree was dirty at run time
- Fail loudly and specifically on an unknown key, rather than ignoring it

Concepts to learn

- `tomllib` in the standard library: <https://docs.python.org/3/library/tomllib.html>
- Schema validation with `pydantic`: <https://docs.pydantic.dev/>
- Hashing: <https://docs.python.org/3/library/hashlib.html>
- Why configuration should be data and not code

Where to find the material

- The Turing Way, on reproducible research practice: <https://the-turing-way.netlify.app/>
- Hydra, for a view of what a mature configuration system looks like: <https://hydra.cc/>

Definition of done

- A run of project 7 is fully driven by a config file, with no hardcoded parameters
- Rerunning with the same config produces byte-identical output, verified by a test
- An unknown key produces an error naming the key and listing the valid ones
- The output directory contains the resolved config and the git state, and you can reconstruct the run from them alone
- A parameter sweep of ten runs works from the command line without editing any file

Edge cases to handle

- A config file that does not exist, or is malformed
- A value of the right type but an invalid range, such as a negative timestep
- Nested configuration sections
- An override for a key that does not exist in the schema
- Running outside a git repository, where the commit hash is unavailable
- Two different configs that hash to the same directory name, which should be impossible but must be handled

Project 10: Command line framework for your scientific tools

What you are building

A consistent command line interface for evkit, with subcommands, useful help text, and correct exit codes, so that every tool you build is invocable the same way.

Why it matters

Scripts you run by editing the bottom of the file are not tools. This is what turns your toolkit into something you can call from a shell script, a cron job, or a cluster batch submission in project 193. It replaces the calculator and unit converter projects from the original curriculum and teaches the same argument handling in a form you will actually use.

Requirements

- A single entry point, `ev`, with subcommands: `ev tle`, `ev time`, `ev horizons`, `ev telemetry`
- Built with `argparse` and `subparsers`. Resist reaching for a framework until you have done it by hand once
- Every subcommand has help text with at least one worked example
- Global options that apply to all subcommands: verbosity, config file, output directory
- Correct exit codes: 0 for success, 1 for a handled error, 2 for a usage error
- Errors go to `stderr`, results go to `stdout`, so the tool composes with pipes
- Reads from `stdin` when no input file is given, where that makes sense
- Registered as a console script in `pyproject.toml` so `ev` works after `pip install`

Concepts to learn

- `argparse` and `subparsers`: <https://docs.python.org/3/library/argparse.html>
- Exit codes and `sys.exit` conventions: <https://docs.python.org/3/library/sys.html#sys.exit>
- `stdout` versus `stderr`, and why mixing them breaks piping
- Console script entry points: <https://packaging.python.org/en/latest/specifications/entry-points/>

Where to find the material

- The `argparse` tutorial: <https://docs.python.org/3/howto/argparse.html>
- Command Line Interface Guidelines, which is short and unusually good: <https://clig.dev/>
- Click and Typer, as alternatives to look at after you have built it with `argparse`: <https://typer.tiangolo.com/>

Definition of done

- `ev tle parse file.tle | ev tle filter --inclination 95:100` works as a pipeline
- `ev --help` and every `ev` subcommand `--help` are useful without reading the source
- A deliberate misuse returns exit code 2 and a message naming what was wrong
- Installed via `pip install -e` and callable as `ev` from anywhere
- A shell script in `scripts/` uses it end to end

Edge cases to handle

- No arguments at all, which should print help rather than a traceback
- Conflicting options
- Output to a file that already exists. Refuse, or require an explicit overwrite flag
- Broken pipe, which happens when you pipe into `head` and is a real error you must handle
- Unicode in file paths and arguments
- Interrupt with `Ctrl-C`, which should exit cleanly rather than dumping a traceback

Project 11: Test suite for the units and time libraries

What you are building

A comprehensive pytest suite covering projects 1 through 4, with fixtures, parametrisation, and proper handling of floating point comparison.

Why it matters

This is the project where your toolkit becomes trustworthy. Everything after this depends on projects 1 to 10 being correct, and the only way you can know that is a test suite you believe in. It is also rule three of the five, applied for the first time.

Requirements

- At least eighty tests across units, constants, time and frames
- Parametrised tests so that adding a new unit or a new time scale does not mean writing a new test function
- Fixtures for shared setup, such as a loaded leap second table or a sample catalogue
- Floating point comparison with `pytest.approx` or explicit tolerances. Never use equality on floats
- Reference value tests: at least ten cases checked against published external values, cited in the test docstring
- Round-trip tests: convert and convert back, assert the error is below tolerance
- Property tests: for any valid input, dimensions are preserved under multiplication
- Error path tests: every custom exception is raised by at least one test
- Coverage measured and reported, with a target of ninety percent on `evkit`

Concepts to learn

- pytest basics: <https://docs.pytest.org/en/stable/getting-started.html>
- Fixtures: <https://docs.pytest.org/en/stable/how-to/fixtures.html>
- Parametrisation: <https://docs.pytest.org/en/stable/how-to/parametrize.html>
- `approx` and floating point comparison: <https://docs.pytest.org/en/stable/reference/reference.html#pytest-approx>
- Coverage: <https://coverage.readthedocs.io/>

Where to find the material

- The pytest documentation, which is worth reading in full once
- Effective Python by Brett Slatkin, items on testing
- Look at how astropy tests its units module, in the astropy source tree. Reading a mature test suite teaches more than any tutorial

Definition of done

- pytest passes with zero failures and zero warnings
- Coverage on `evkit` is at least ninety percent, and you can explain what the uncovered lines are
- The suite runs in under ten seconds, so you actually run it
- At least one test that currently fails is written first, then made to pass, so you know the tests can fail
- Reference value tests cite their sources in docstrings

Edge cases to handle

- Comparing quantities with different but compatible units
- Tests that depend on network access, which must be marked and skippable
- Tests that depend on a data file, which must fail clearly if the file is missing
- Platform-dependent floating point behaviour

- Tests that pass for the wrong reason, which is why you write a failing test first

Project 12: Numerical assertion helpers and tolerance utilities

What you are building

A small module of comparison functions for floating point values, with explicit absolute and relative tolerances, ULP-based comparison, and clear failure messages.

Why it matters

Every validation plot you make for the next three years rests on a comparison, and every comparison rests on a tolerance. This is the module that makes you fluent in why 0.1 plus 0.2 is not 0.3, and it is a direct prerequisite for the floating point lab in project 50 and every verification project in Phase 2.

Requirements

- `assert_close` with independent absolute and relative tolerances, and a failure message that reports both the absolute and relative error achieved
- Comparison in units in the last place, using `math.ulp`
- Array comparison that reports the index and magnitude of the worst disagreement, not just that it failed
- A function that reports the number of significant digits two values agree to
- Detection and clear reporting of NaN and infinity, which should never silently compare equal
- A relative error function that behaves sensibly when the reference is zero
- Integration with `pytest` so failures are readable

Concepts to learn

- Floating point representation: <https://docs.python.org/3/tutorial/float.html>
- What Every Computer Scientist Should Know About Floating-Point Arithmetic, by Goldberg. Read at least the first third: https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html
- `math.ulp` and `math.isclose`: <https://docs.python.org/3/library/math.html>
- Machine epsilon and `sys.float_info`: https://docs.python.org/3/library/sys.html#sys.float_info

Where to find the material

- `numpy.testing`, as a design reference: <https://numpy.org/doc/stable/reference/routines.testing.html>
- The Goldberg paper above is the canonical reference and worth returning to repeatedly

Definition of done

- Used by the test suite from project 11 onward, replacing bare `pytest.approx` where more detail is useful
- A failure message tells you the achieved absolute error, relative error, and ULP distance, so you can decide whether the tolerance or the code is wrong
- Handles a reference value of exactly zero without dividing by zero
- Documented with a short note on when to use absolute versus relative tolerance, which is a genuinely subtle question

Edge cases to handle

- Reference value of zero, where relative error is undefined
- Both values NaN, which are never equal but might be the expected outcome
- Positive and negative zero
- Denormal numbers near the underflow limit
- Very large values where absolute tolerance is meaningless
- Arrays of different shapes

Project 13: Bash toolkit for scientific workflows

What you are building

A set of shell scripts that automate the repetitive parts of running, archiving and syncing computational work.

Why it matters

Every cluster you will ever use is driven by a shell. This is groundwork for project 193, SLURM job submission, and it is the difference between running an experiment and running fifty experiments overnight.

Requirements

- `run_sweep.sh`: run a parameter sweep by invoking your ev tool with a list of configs, logging each run
- `archive_results.sh`: tar and timestamp an output directory, with a manifest of what it contains
- `sync_results.sh`: rsync results to a backup location, with a dry run mode
- `env_check.sh`: verify that the tools a project needs are installed and report versions
- `clean_runs.sh`: remove output directories older than N days, with confirmation
- `watch_run.sh`: tail a running log and alert when it finishes or errors
- Every script starts with `set -euo pipefail` and has a usage function
- Every script handles being called with no arguments by printing usage and exiting 2

Concepts to learn

- Bash strict mode and why `set -euo pipefail` matters: <http://redsymbol.net/articles/unofficial-bash-strict-mode/>
- Bash manual: <https://www.gnu.org/software/bash/manual/bash.html>
- rsync: <https://download.samba.org/pub/rsync/rsync.1>
- Quoting in shell, which is the single largest source of shell bugs
- shellcheck, which will teach you more about bash than any tutorial: <https://www.shellcheck.net/>

Where to find the material

- The ShellCheck wiki, which explains each warning it emits and is an excellent bash course by accident
- Greg's Bash FAQ: <https://mywiki.woledge.org/BashFAQ>

Definition of done

- Every script passes shellcheck with no warnings
- A sweep of ten configurations runs unattended and logs each one separately
- The archive script produces something you can hand to someone else and they can understand
- Scripts work with paths containing spaces, which is the standard test of whether your quoting is right
- All scripts are executable, documented in a `scripts/README`, and used at least once for real

Edge cases to handle

- Paths containing spaces or unicode
- A command failing midway through a sweep. Continue or abort, but decide and document which
- Disk full during archiving
- Being run from a different working directory than expected
- Being run on macOS versus Linux, where `sed` and `date` differ

Project 14: Linux environment built from scratch

What you are building

A documented, reproducible development environment: shell configuration, dotfiles under version control, SSH keys, and a working understanding of what the PATH actually is.

Why it matters

ICTP MHPC lists solid Linux knowledge as an entry requirement, and every HPC system you will ever touch is a Linux box you reach over SSH. This project also fixes the SSH key problems you have hit before, permanently, by making you understand rather than work around them.

Requirements

- A dotfiles repository under git: `.bashrc` or `.zshrc`, `.gitconfig`, `.vimrc` or editor config, `.ssh/config`
- An install script that symlinks dotfiles into place on a fresh machine
- SSH key pairs generated properly, with ed25519, passphrase protected, and an agent configured
- An `.ssh/config` with named host entries, so you never type an IP address again
- Documented understanding of PATH, and how your shell resolves a command
- A tool version manager for Python: `pyenv` or `conda`, chosen deliberately with the reason written down
- A written note in `docs/decisions` explaining every non-obvious choice
- Test it: bring up a fresh container or VM, run the install script, and confirm you have a working environment

Concepts to learn

- The Linux filesystem hierarchy: https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html
- How the shell finds commands, and what PATH order means
- SSH key authentication: <https://www.ssh.com/academy/ssh/public-key-authentication>
- Environment variables and shell startup files, and which of `.profile`, `.bashrc` and `.bash_profile` runs when
- File permissions and why SSH refuses keys with the wrong mode

Where to find the material

- The Missing Semester of Your CS Education, MIT, which is the best short Linux and tooling course available: <https://missing.csail.mit.edu/>
- The Arch Wiki, which is the best Linux documentation on the internet regardless of your distribution: <https://wiki.archlinux.org/>

Definition of done

- A fresh container reaches a working environment with one command
- You can SSH to a remote host by name with no password and no key errors
- You can explain, without looking it up, why `.bashrc` is not read by a login shell
- Dotfiles are on GitHub and you have restored from them at least once
- The decisions note explains your Python version management choice and what you rejected

Edge cases to handle

- Key permissions that are too open, which SSH silently refuses
- Login versus non-login versus interactive shells, and the different startup files each reads
- PATH ordering, where a system Python shadows the one you intended
- Different behaviour between `bash` and `zsh`
- Dotfiles that assume a tool is installed and break the shell when it is not

Project 15: Git fluency drill

What you are building

A deliberate practice repository where you rehearse the git operations that matter under pressure: interactive rebase, bisect, conflict resolution, reflog recovery, and history rewriting.

Why it matters

You already use git daily, but commit and push is perhaps a fifth of it. Every open source contribution in project 293 will require a clean rebased branch and a coherent commit history. This is the difference between a maintainer merging your work and a maintainer asking you to redo it.

Requirements

- Create a practice repository with deliberately messy history
- Interactive rebase: squash, reword, reorder, drop, and split a commit
- Resolve a genuine three-way merge conflict by hand, and understand what the markers mean
- Use git bisect to find a commit that introduced a bug, with a script for automated bisection
- Recover a deleted branch and a lost commit using the reflog
- Use git stash, cherry-pick, and revert, and know when each is appropriate
- Configure and use a .gitignore and .gitattributes properly
- Write commit messages in the conventional form: imperative subject under 50 characters, blank line, body explaining why not what
- Sign your commits with a GPG or SSH key
- Write up what you learned in docs/decisions as a personal reference

Concepts to learn

- The git book, chapters 3, 6 and 7: <https://git-scm.com/book/en/v2>
- Interactive rebase: <https://git-scm.com/docs/git-rebase>
- git bisect: <https://git-scm.com/docs/git-bisect>
- The reflog, which is the safety net that means you can never really lose work: <https://git-scm.com/docs/git-reflog>
- How to write a git commit message: <https://cbea.ms/git-commit/>

Where to find the material

- Learn Git Branching, an interactive visual trainer: <https://learngitbranching.js.org/>
- Oh Shit, Git, for the situations you will actually hit: <https://ohshitgit.com/>
- Read the contributing guide of astropy or numpy, since their workflow is what you will need in project 293

Definition of done

- You have used bisect to find a real bug in your own toolkit history, and can describe the story
- You can rebase a five-commit branch into two clean commits without looking up the syntax
- You have recovered a commit you deliberately destroyed, using only the reflog
- Your evkit history has meaningful commit messages you would not be embarrassed to have reviewed
- Commits are signed and show as verified on GitHub

Edge cases to handle

- A conflict in a binary file, which cannot be merged by hand
- Rebasing a branch that has already been pushed, and why that is dangerous for shared branches
- A detached HEAD state, and how to get out of it

- Line ending differences between platforms
- Large files committed by accident, and how to remove them from history
- Credentials or keys committed by accident, which must be treated as compromised, not merely removed

Project 16: Data downloader with caching, retries and rate limits

What you are building

A robust HTTP client for scientific data sources, with on-disk caching, exponential backoff on failure, rate limiting, and integrity verification.

Why it matters

Celestrak, Horizons, MAST, Copernicus and NOAA all sit behind HTTP, and all of them will rate limit you, time out, or return a stale file at some point. This module replaces the API and web scraper projects from the original curriculum and becomes the front door to every real dataset you use from here on.

Requirements

- GET with configurable timeout, and a descriptive User-Agent identifying you and the project
- Exponential backoff with jitter on 429, 500, 502, 503 and 504, with a maximum retry count
- Respect the Retry-After header when the server sends one
- On-disk cache keyed by URL hash, with a configurable time to live
- Conditional requests using ETag and If-Modified-Since, so you re-download only when the file changed
- Rate limiting: a minimum interval between requests to the same host
- Checksum verification where the source publishes one
- Streaming download for large files, so a two gigabyte scene does not go through memory
- Resume interrupted downloads using range requests
- Convenience wrappers for the sources you will use: Celestrak, Horizons, NOAA SWPC

Concepts to learn

- HTTP status codes: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- The requests library: <https://requests.readthedocs.io/>
- Exponential backoff and jitter: <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>
- Conditional requests and ETags: https://developer.mozilla.org/en-US/docs/Web/HTTP/Conditional_requests
- Streaming responses and chunked transfer

Where to find the material

- The requests documentation, particularly the advanced usage page
- urllib3 retry configuration: <https://urllib3.readthedocs.io/en/stable/reference/urllib3.util.html>
- Read the terms of use of Celestrak and Space-Track before you automate against them. Being blocked is avoidable and permanent embarrassment

Definition of done

- Fetches the full Celestrak active satellite catalogue, caches it, and a second call within the TTL makes no network request
- A simulated 429 response triggers backoff with the right delays, verified by a test with a mocked server
- A large file downloads by streaming and the process memory stays flat
- An interrupted download resumes rather than restarting
- Every test in your suite runs offline against the cache, so CI does not depend on external services
- Rate limiting is verifiably respected, with a test measuring inter-request intervals

Edge cases to handle

- No network at all, which must fail with a clear message rather than a raw exception

- DNS failure versus connection refused versus timeout, which are different problems
- A server returning HTML error pages with status 200
- Redirects, including redirect loops
- Cache corruption, where a partially written file must not be treated as valid
- Clock skew affecting TTL logic
- Sources requiring authentication, such as Space-Track, where credentials must come from the environment and never from source code

Project 17: HDF5 and NetCDF reader and writer

What you are building

A module for writing and reading structured scientific output in HDF5 and NetCDF, with chunking, compression, and self-describing metadata.

Why it matters

Simulation output is never CSV. When your N-body run in project 33 produces a million timesteps of particle state, or your Earth observation pipeline in project 230 handles a multi-band time series, text files stop being viable. Learning the real formats now means you never have to retrofit them later.

Requirements

- Write a simulation result as HDF5: datasets for each variable, groups for logical structure, attributes for metadata
- Store the configuration hash and git commit from project 9 as file-level attributes, so any output file is traceable to the run that made it
- Chunking chosen deliberately for the expected access pattern, with the reasoning documented
- Compression with gzip or lz4, with a measurement of the size and speed trade-off
- Append to an existing file incrementally, so a long run does not hold everything in memory
- Read back partially: load one variable, or a slice of one variable, without touching the rest
- The same for NetCDF, and a note on when NetCDF's conventions are preferable to raw HDF5
- Follow the CF conventions for units and coordinate variables where they apply

Concepts to learn

- HDF5 concepts, groups, datasets, attributes: <https://docs.h5py.org/en/stable/quick.html>
- Chunking and its performance implications: <https://docs.h5py.org/en/stable/high/dataset.html#chunked-storage>
- NetCDF and the CF conventions: <https://cfconventions.org/>
- xarray, which sits on top of NetCDF and which you will use heavily in Phase 9: <https://docs.xarray.dev/>

Where to find the material

- The h5py documentation: <https://docs.h5py.org/>
- The HDF Group's tutorials: <https://portal.hdfgroup.org/>
- HDFView, a GUI for inspecting files, which is useful for confirming your structure is sensible

Definition of done

- Writes a file that someone else can open and understand without your code, verified with h5dump or HDFView
- A slice read from a one gigabyte file is fast and does not load the whole file, timed and reported
- Chunk shape choice is documented with a measurement showing it beats the default for your access pattern
- Compression ratio and write time overhead are measured and recorded
- Round-trip preserves every value bit for bit, verified by tests

Edge cases to handle

- Very large datasets exceeding memory
- Variable-length strings, which HDF5 handles awkwardly
- Unicode in attribute values
- A file left open by a crashed process
- Concurrent read while another process writes
- Chunk shapes that do not divide the dataset shape evenly

- NaN and infinity values, which survive HDF5 but may confuse downstream tools

Project 18: FITS file reader and header explorer

What you are building

A reader for the Flexible Image Transport System, the standard format for astronomical data, including header parsing, image and table extension handling, and a world coordinate system summary.

Why it matters

FITS is how essentially all astronomical data is distributed, and it has been since 1981. Every project in Phase 11, and the Kepler and Gaia work in particular, starts by opening a FITS file. Understanding the format directly, rather than only through astropy, means you can diagnose a malformed file instead of being stopped by one.

Requirements

- Open a FITS file and enumerate its header data units
- Parse header cards into a structured mapping, preserving comments and card order
- Handle the mandatory keywords: SIMPLE, BITPIX, NAXIS, and the extension keywords
- Read image data into an array with the correct dtype from BITPIX, applying BSCALE and BZERO
- Read binary table extensions, including column names, formats and units
- Summarise the world coordinate system keywords and explain what projection the file uses
- Handle multi-extension files, which is the normal case for real survey data
- A pretty-printed header dump for inspection from the command line, wired into ev from project 10

Concepts to learn

- The FITS standard, which is short and readable: https://fits.gsfc.nasa.gov/fits_standard.html
- astropy.io.fits: <https://docs.astropy.org/en/stable/io/fits/>
- World coordinate systems: <https://docs.astropy.org/en/stable/wcs/>
- BITPIX, BSCALE and BZERO, and why integer FITS data is often scaled

Where to find the material

- The FITS support office at NASA GSFC: <https://fits.gsfc.nasa.gov/>
- MAST, for real files to test against: <https://mast.stsci.edu/>
- A Kepler or TESS light curve file is an excellent test case and connects directly to project 277

Definition of done

- Opens a real Kepler or TESS light curve file from MAST and correctly reports every extension
- Your parsed header matches astropy.io.fits card for card on at least five real files
- Image data with nontrivial BSCALE and BZERO reads back to the correct physical values
- A binary table extension is read with correct column types and units
- The header dump is wired into your CLI and is genuinely useful for inspecting an unfamiliar file

Edge cases to handle

- Files with non-standard or missing mandatory keywords, which are common in older archives
- HIERARCH cards for keywords longer than eight characters
- CONTINUE cards for long string values
- Compressed image extensions, which are a different structure
- Files with a data section that does not match the declared dimensions
- Very large images that should be memory-mapped rather than loaded
- Non-ASCII characters in headers, which are technically forbidden and nonetheless appear

When Phase 0 is finished

You will know Phase 0 is complete when all of the following are true, not when eighteen boxes are ticked.

- evkit installs with `pip install -e` and imports cleanly on a fresh machine.
- pytest passes with at least ninety percent coverage and runs in under ten seconds.
- The `ev` command line tool works from any directory and its help text is genuinely useful.
- You can fetch a live TLE from Celestrak, parse it, convert its epoch to four time scales, and report the satellite's position in three coordinate frames, using only your own code.
- You can fetch an Earth ephemeris from JPL Horizons and interpolate a position at an arbitrary instant.
- Every module has a docstring you would not be embarrassed to have reviewed.
- The build log in the README has an entry for every week of the phase, including the bad ones.

At that point you have something no tutorial portfolio has: a tested, installable, documented scientific library that you wrote yourself and that everything after this depends on. It is also, incidentally, the first candidate for the PyPI release in project 81 and the JOSS submission in project 296.

Specifications for Phase 1, projects 19 to 46, the compiled languages block, should be written when you are roughly two thirds through this phase. Writing them now would mean writing them against an understanding of the problem that you do not have yet, and they would be worse for it.